



LINKWIZ

Skill Exchange Platform

PROJECT REPORT

Members Name and Roll Numbers:

Member 1: Maira Malik

Roll No: 2023-BSE-040

Member 2: Sana Tariq

Roll No: 2023-BSE-058

Member 3: Reena Qureshi

Roll No: 2023-BSE-052

Semester: VI-B

Submitted To: Dr. Aliyah Ashraf

Subject: Web Engineering

Institution: Fatima Jinnah Women University

Abstract

LinkWiz is a web-based skill exchange platform designed to connect individuals who wish to learn new skills by exchanging knowledge with others. The system enables users to create profiles, list offered skills, specify desired skills, send exchange requests, communicate through real-time messaging, and manage ongoing collaborations.

The platform was developed using the MERN Stack, which includes MongoDB, Express.js, React.js, and Node.js. The primary objective is to create a collaborative environment where users can share expertise without requiring monetary transactions. Unlike traditional tutoring platforms or paid courses, LinkWiz operates on a barter model for skills. For example, a student who knows Python can exchange lessons with someone who plays guitar, while a graphic designer can swap sessions with a web developer. The platform removes financial barriers from education and makes learning collaborative, community-driven, and free.

LinkWiz provides authentication, profile management, smart skill matching, request lifecycle management, real-time messaging, review and rating features, and dashboard analytics, all within a clean, responsive interface built with Tailwind CSS.

1. Introduction

1.1 Background

The increasing demand for continuous learning has created a need for platforms that allow individuals to exchange knowledge and skills freely. Traditional learning platforms often require significant financial investment, making them inaccessible for many users, particularly students and early-career professionals. University students, young professionals, and freelancers often possess rich and valuable knowledge but lack the budget to pay for premium courses or expert tutors.

LinkWiz addresses this problem by enabling peer-to-peer skill exchange, democratizing education through a collaborative barter system where no money changes hands, only knowledge.

1.2 Problem Statement

Many individuals possess valuable skills but lack access to the resources required to learn new ones. Existing platforms focus primarily on paid learning services such as Udemy, Coursera, and Skillshare, requiring paid subscriptions and offering limited peer-to-peer interaction. There is no direct skill exchange mechanism available that connects learners mutually. A system is therefore needed where users can exchange skills directly and benefit from each other without monetary constraints.

1.3 Objectives

- Develop a comprehensive, full-stack web-based skill exchange platform.
- Enable secure user authentication and profile management.
- Allow users to create and manage profiles with offered and desired skills.
- Support a smart matching algorithm to connect compatible users.
- Implement exchange request lifecycle management: send, accept, reject, complete.
- Facilitate real-time communication between matched users via Socket.io.
- Track active, pending, and completed exchanges through a personal dashboard.
- Provide a rating and review system to build community accountability.
- Include safety features such as user reporting, blocking, and moderation tools.

1.4 Target Audience

User Group	Description
University Students	Students with limited budgets but rich programming, design, or academic skills to share.
Young Professionals	Early-career individuals seeking to upskill without expensive bootcamps.
Freelancers	Professionals looking to expand their service offerings collaboratively.
Lifelong Learners	Individuals of all ages seeking continuous, flexible learning opportunities.
Geographic Focus	Pakistan, designed with scalability to other regions in mind.

2. System Analysis

2.1 Existing System

Current platforms dominate the online education space but fail to facilitate direct peer-to-peer skill bartering.

- Udemy and Coursera require paid subscriptions or per-course fees.
- Skillshare operates on a subscription model with no peer exchange mechanism.
- All existing platforms follow a one-to-many model, instructor to many students, with no mutual skill exchange.
- There is no community-driven, barter-based learning ecosystem available in Pakistan.

2.2 Proposed System

LinkWiz introduces a collaborative ecosystem where users exchange skills instead of paying for courses.

Feature	LinkWiz Advantage
Cost	Completely free skill barter replaces money.
Interaction Model	One-to-one mutual exchange, not one-to-many broadcasting.
Community	Users build networks and rate each other for accountability.
Accessibility	No subscription needed; open to all skill levels.
Local Focus	Built for Pakistani users with city-based matching.

2.3 Customer Requirements / User Personas

Requirements were gathered through storytelling-based analysis using five key user personas.

2.3.1 Sarah Ahmed, The University Student

Background: 20-year-old CS student at NUST, Islamabad. Strong Python skills, wants to learn guitar, has a very limited monthly budget.

Requirements extracted:

- Easy profile creation with skill selection dropdowns.
- Smart matching algorithm to find compatible users quickly.
- In-platform communication without sharing personal contact details.
- Session coordination tools within the platform.

2.3.2 Ahmed Raza, The Young Professional

Background: 26-year-old Marketing Executive wanting to transition into UX Design. Bootcamps cost 50,000–100,000 PKR, which is unaffordable.

Requirements extracted:

- Skill exchange request system with clear request lifecycle.
- Profile showcasing depth of expertise to build credibility.
- Rating and review system so users can trust each other.
- Exchange history tracking to document completed sessions.

2.3.3 Fatima Khan, The Freelancer

Background: 24-year-old Graphic Designer wanting to expand into web development to offer more services to clients.

Requirements extracted:

- Barter-style exchange system with clearly defined session counts.
- Session tracking per exchange to ensure fair value.
- Rating and review mechanism to hold both parties accountable.

2.3.4 Dr. Tariq, The Lifelong Learner

Background: 55-year-old university professor wanting to learn video editing for his online course content.

Requirements extracted:

- Age-inclusive, simple, and intuitive interface.
- Simple search functionality to find users by skill.
- Flexible scheduling with no rigid session timings enforced by the platform.

2.3.5 Platform Moderator, Safety Perspective

The moderation persona ensures the platform remains safe for all users.

Requirements extracted:

- Message logging and monitoring capabilities.
- User reporting system for flagging inappropriate behavior.
- Admin-level moderation tools including user suspension.

3. Functional Requirements

ID	Module	Requirement
FR-01	User Management	Users must be able to register with name, email, password, city, and bio.
FR-02	User Management	Users must be able to log in and log out securely.
FR-03	User Management	Users must be able to edit their profile information at any time.
FR-04	Skills	Users must be able to add and remove skills they offer.
FR-05	Skills	Users must be able to add and remove skills they want to learn.
FR-06	Matching	The system must match users based on mutual skill interests using a scoring algorithm.
FR-07	Exchange	Users must be able to send, accept, decline, and complete exchange requests.

ID	Module	Requirement
FR-08	Exchange	Exchange status must progress through Pending, Accepted, Completed.
FR-09	Messaging	Matched users must be able to communicate via real-time in-platform chat.
FR-10	Messaging	Chat history must be persisted and retrievable.
FR-11	Dashboard	Users must see all active, pending, and completed exchanges in a personal dashboard.
FR-12	Reviews	Users must be able to rate and review each other after exchange completion.
FR-13	Search	Users must be able to search for others by skill, city, or availability.
FR-14	Safety	Users must be able to report and block other users.
FR-15	Admin	Platform moderators must be able to view reports and manage users.

4. Non-Functional Requirements

Category	Requirement	Implementation
Performance	Fast API responses, efficient DB queries.	MongoDB indexing on userId, skillId, exchangeId fields.
Security	Secure authentication and data protection.	JWT with httpOnly cookies, bcrypt hashing, helmet.js, CORS configuration.
Usability	Responsive, user-friendly interface.	Tailwind CSS utility-first framework, React Router for smooth navigation.
Reliability	Stable backend with error handling.	Global Express error handlers, Mongoose validation, structured error responses.
Scalability	System should handle growing user base.	Rate limiting, efficient Socket.io rooms, MongoDB Atlas cloud scaling.
XSS Protection	Prevent cross-site scripting attacks.	React escapes output by default, helmet.js security headers applied.
Input Validation	Reject malformed or malicious input.	express-validator with sanitization on all POST/PUT endpoints.

5. System Design

5.1 Three-Tier Architecture

LinkWiz follows a modern three-tier architecture separating concerns into three distinct layers.

Layer	Technology	Responsibility
Presentation Layer	React.js, Tailwind CSS, Axios, React Router	Renders UI, handles user interaction, manages client-side routing, sends HTTP requests to backend.
Application Layer	Node.js, Express.js, Socket.io, JWT, bcrypt	Processes business logic, handles authentication middleware, manages real-time events, serves REST API.
Data Layer	MongoDB Atlas, Mongoose ODM	Stores and retrieves user profiles, skills, exchanges, messages, reviews, and reports.

5.2 Data Flow

The data flow through the system follows a structured request-response cycle.

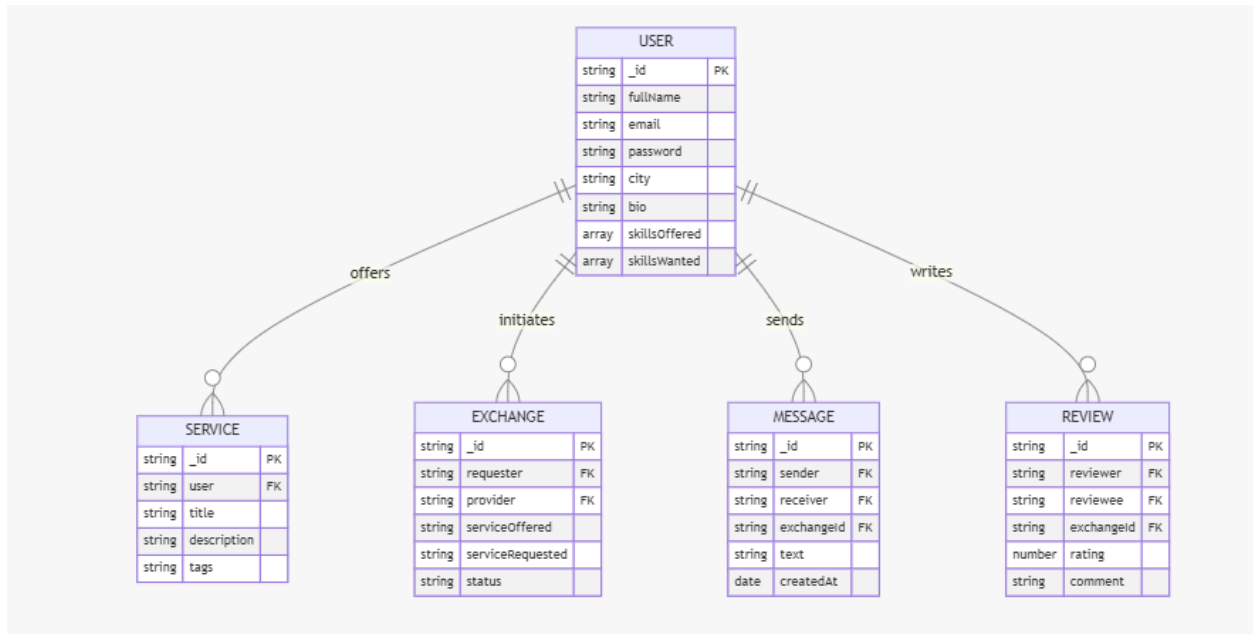
1. User performs an action in the browser, such as clicking Send Exchange Request.
2. React component triggers an Axios HTTP request to the backend API.
3. Express middleware validates the JWT token in the request.
4. Route handler processes the business logic.
5. Mongoose query executes against MongoDB Atlas.
6. JSON response is returned to the frontend.
7. React state updates and the UI re-renders with new data.
8. For messaging events, Socket.io emits events directly between connected clients without polling.

5.3 Database Schema Design

LinkWiz uses MongoDB with the following collection structure.

Collection	Key Fields	Description
users	id, name, email, password, city, bio, avatarUrl, createdAt	Stores all user profile data. Password stored as bcrypt hash.
skills	id, name, category	Master list of all available skills on the platform.
userskills	userId, skillId, type, proficiency, priority	Junction collection linking users to their skills with metadata.
exchanges	id, matchId, initiatorId, recipientId, status, proposedSessions, scheduledDates, createdAt, completedAt	Records each skill exchange and its lifecycle status.

Collection	Key Fields	Description
messages	id, exchangeId, senderId, content, isRead, timestamp	Stores all chat messages scoped to an exchange.
reviews	id, exchangeId, reviewerId, revieweeId, rating, comment, createdAt	Post-exchange ratings and comments.
reports	id, reporterId, reportedId, reason, status, createdAt	User-submitted safety reports for moderation.



5.4 Smart Matching Algorithm

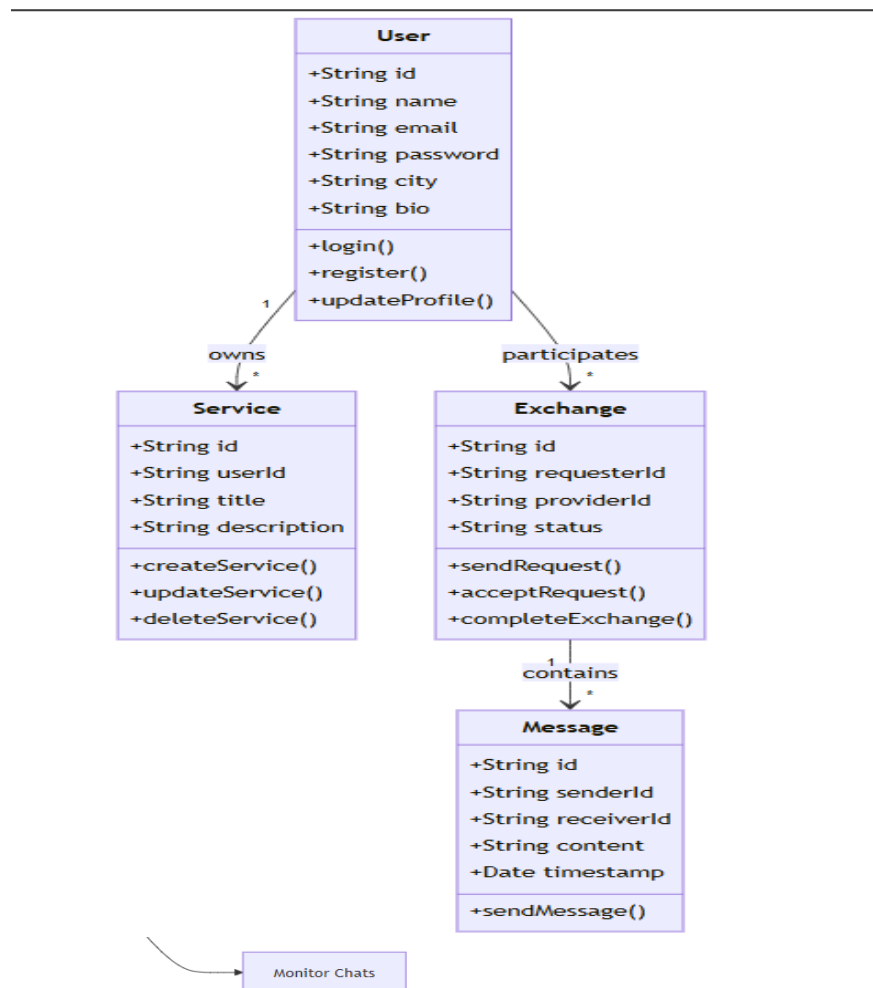
The matching algorithm connects users based on mutual skill compatibility.

1. Retrieve the current user's offered skills, e.g., Python, JavaScript.
2. Retrieve the current user's wanted skills, e.g., Guitar, Public Speaking.
3. Query the database to find all other users whose offered skills overlap with the current user's wanted skills and whose wanted skills overlap with the current user's offered skills.
4. For each candidate user found, calculate a match score based on the number of mutual overlaps, proficiency levels, and location proximity; same city scores higher.
5. Sort candidates by match score in descending order.
6. Return the ranked list of match cards to the frontend dashboard.

5.5 Use Case Summary

Actor	Use Case	Description
User Guest	Register	Create a new account with profile details.
User Guest	Login	Authenticate and receive a JWT session token.
Registered User	Create Profile	Add bio, city, avatar, and skill preferences.
Registered User	View Matches	See a list of compatible users based on mutual skills.
Registered User	Send Request	Initiate a skill exchange request to another user.
Registered User	Accept/Decline	Respond to incoming exchange requests.
Registered User	Chat	Communicate in real-time with exchange partner.
Registered User	Rate User	Submit a rating and review after exchange completion.
Registered User	Report User	Flag inappropriate behavior to platform moderators.
Admin	Manage Users	Review reports and suspend or ban offending accounts.

5.6 Class Diagram Key Entities



Class	Attributes	Methods
User	id, name, email, password, city, bio, avatarUrl, createdAt	register, login, updateProfile, getProfile
Skill	id, name, category	create, findAll, findByCategory
UserSkill	userId, skillId, type, proficiency, priority	add, remove
Match	id, userAId, userBId, matchScore	generateMatches, calculateMatchScore
Exchange	id, matchId, initiatorId, recipientId, status, proposedSessions	createRequest, accept, decline, complete
Message	id, exchangeId, senderId, content, isRead, timestamp	sendMessage, getMessages
Review	id, exchangeId, reviewerId, revieweeId, rating, comment	createReview, getAverageRating

6. Technology Stack

Layer	Technology	Version	Purpose
Frontend	React.js	18.x	Component-based UI development with virtual DOM.
Frontend	Tailwind CSS	3.x	Utility-first CSS framework for responsive design.
Frontend	Axios	1.x	Promise-based HTTP client for API requests.
Frontend	React Router DOM	6.x	Client-side navigation and protected route management.
Backend	Node.js	18.x	JavaScript runtime for server-side execution.
Backend	Express.js	4.x	Minimal, fast REST API framework.
Backend	Socket.io	4.x	WebSocket-based real-time bidirectional communication.
Database	MongoDB	6.x	Flexible NoSQL document database.
Database	Mongoose	7.x	Object Data Modeling with schema validation.
Auth	JSON Web Token	9.x	Stateless token-based authentication.
Auth	bcrypt.js	5.x	Password hashing with configurable salt rounds.
Hosting	Vercel	-	Frontend deployment with global CDN.
Hosting	Render	-	Backend Node.js server deployment.
Hosting	MongoDB Atlas	-	Managed cloud database with IP whitelisting.
Dev Tools	Postman	-	API endpoint testing and documentation.

Layer	Technology	Version	Purpose
Dev Tools	GitHub	-	Version control and team collaboration.

7. Implementation

7.1 Authentication Module

Users register and log in securely using JWT-based authentication. Passwords are hashed using bcrypt salt rounds 10 before being stored in MongoDB. On login, the server generates a JWT token stored in an httpOnly cookie to prevent XSS attacks. Backend middleware intercepts all protected requests and verifies token validity before allowing access.

Method	Endpoint	Description
POST	/api/auth/register	Validates input, hashes password, creates user record, returns JWT.
POST	/api/auth/login	Verifies email and password hash, returns JWT in httpOnly cookie.
POST	/api/auth/logout	Clears the JWT cookie and ends the session.

7.2 User Management Module

Users can update their personal information, including name, city, bio, and avatar, and manage their skill sets at any time. Skills are stored in a separate collection and linked via the userskills junction collection, allowing flexible proficiency ratings and priorities.

Method	Endpoint	Description
GET	/api/users/me	Returns the authenticated user's full profile.
PUT	/api/users/me	Updates profile fields name, city, bio, avatarUrl.
GET	/api/users/:id	Returns another user's public profile.
GET	/api/users/search	Searches users by skill, city, or name.
POST	/api/users/me/skills/offered	Adds a skill to the user's offered skills list.
DELETE	/api/users/me/skills/offered/:id	Removes a skill from the user's offered skills.
POST	/api/users/me/skills/wanted	Adds a skill to the user's wanted skills list.

7.3 Skill Matching Module

When a user views their dashboard, the platform queries the matching engine. The algorithm cross-references the current user's offered skills against other users' wanted skills, and vice versa, to find mutual matches. Each candidate is assigned a numeric match score and the results are returned sorted from highest to lowest score, displayed as match cards on the frontend.

Method	Endpoint	Description
GET	/api/matches	Returns a ranked list of compatible users for the authenticated user.

7.4 Exchange Request Module

The exchange request lifecycle is the core business process of LinkWiz. A user initiates a request, the recipient reviews and accepts or declines, and upon completion both parties can submit a review.

Method	Endpoint	Description
POST	/api/exchanges/request	Creates a new exchange request with Pending status.
GET	/api/exchanges	Returns all exchanges: active, pending, completed for the user.
GET	/api/exchanges/:id	Returns full details of a specific exchange.
PUT	/api/exchanges/:id/accept	Updates status to Accepted; Socket.io notifies the initiator.
PUT	/api/exchanges/:id/decline	Updates status to Declined; Socket.io notifies the initiator.
PUT	/api/exchanges/:id/complete	Marks exchange as Completed; triggers review prompt.

7.5 Real-Time Messaging Module

Real-time communication is implemented using Socket.io, which maintains persistent WebSocket connections. When two users are matched and an exchange is accepted, they are placed in a shared Socket.io room identified by the exchangeId. Messages are emitted, broadcast within the room, saved to MongoDB for persistence, and rendered on both clients simultaneously without page refresh.

Method	Endpoint	Description
GET	/api/messages/:exchangeId	Retrieves full chat history for a given exchange.

7.6 Reviews and Ratings Module

After an exchange is marked as complete, both users are prompted to rate each other on a scale of 1 to 5 and leave a written comment. Reviews are publicly visible on user profiles and contribute to an aggregate average rating, building trust within the community.

Method	Endpoint	Description
POST	/api/reviews	Creates a new review for a completed exchange.
GET	/api/users/:id/reviews	Returns all reviews and average rating for a user.

7.7 Security Implementation

Security Area	Measure	Implementation Detail
Authentication	JWT with expiration	Tokens stored in httpOnly cookies, not localStorage.
Password Security	bcrypt hashing	Salt rounds 10, passwords never stored in plain text.
Input Validation	express-validator	All POST/PUT routes validate and sanitize inputs.
XSS Protection	React / helmet.js	React escapes by default; security headers applied server-side.
CORS	Configured whitelist	Only the frontend domain is permitted to make API calls.
Rate Limiting	express-rate-limit	100 requests per 15-minute window per IP address.
Database	MongoDB Atlas	IP whitelisting, strong credentials, Atlas-managed backups.

8. Project Management

8.1 Sprint Overview

Sprint	Duration	Theme	Key Deliverables
Sprint 0	Days 1–3	Project Setup	GitHub repo, MongoDB Atlas cluster, Vercel/Render pipelines, folder structure.
Sprint 1	Days 4–10	Auth & Profiles	Registration API, Login API, JWT middleware, profile creation pages, skill management.
Sprint 2	Days 11–17	Matching & Exchanges	Smart matching algorithm, match endpoint, exchange request API, Browse Users page, dashboard.
Sprint 3	Days 18–24	Real-Time Communication	Socket.io server setup, room-based messaging, Chat UI component, unread message notifications.
Sprint 4	Days 25–31	Quality & Safety	Rating and review system, advanced search with filters, report and block user features.
Sprint 5	Days 32–38	Testing & Deployment	Unit and integration tests, cross-browser testing, responsive fixes, final deployment, documentation.

8.2 Resource Allocation

Team Member	Sprint 0	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5
Sana Frontend Lead	50	80	70	60	70	60

Team Member	Sprint 0	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5
Maira Backend Lead	40	80	80	70	70	50
Reena Full Stack	60	40	50	70	60	90

8.3 Project Milestones

Target Day	Description
Day 3	GitHub, MongoDB Atlas, and deployment pipelines fully configured and functional.
Day 10	Users can register, log in, create profiles, and manage skill lists.
Day 17	Matching algorithm operational; users can send and respond to exchange requests.
Day 24	Real-time Socket.io chat fully functional between exchange partners.
Day 31	Review system, search filters, reporting, and blocking features implemented.
Day 38	Platform fully tested, deployed, and documentation/presentation ready.

9. Testing

9.1 Testing Strategy

Testing Level	Tools Used	Coverage Target	Scope
Unit Testing	Jest, React Testing Library	80%	Individual API functions and React components.
Integration Testing	Supertest	70%	Frontend-to-backend API communication and DB interactions.
End-to-End Testing	Cypress	Critical paths	Full user flows: register, match, exchange, chat, review.
Performance Testing	Lighthouse	Score 90+	Page load speed, accessibility, and best practices.

9.2 Test Cases

TC ID	Module	Scenario	Expected Result	Status
TC-01	Authentication	User registers with valid credentials.	Account created; password bcrypt-hashed in DB.	Passed

TC ID	Module	Scenario	Expected Result	Status
TC-02	Authentication	User logs in with correct email and password.	JWT token generated and returned in httpOnly cookie.	Passed
TC-03	Authentication	User accesses a protected route without a token.	401 Unauthorized response returned.	Passed
TC-04	Profile	User updates name and city fields.	Profile updated in MongoDB; response returns updated object.	Passed
TC-05	Skills	User adds Python to offered skills.	Skill linked to user in userskills collection.	Passed
TC-06	Matching	User with Python offered, Guitar wanted views dashboard.	Matched users with Guitar offered returned, sorted by score.	Passed
TC-07	Exchange	User A sends exchange request to User B.	Exchange record created with Pending status.	Passed
TC-08	Exchange	User B accepts the exchange request.	Status updated to Accepted; User A notified via Socket.io.	Passed
TC-09	Messaging	User A sends a message in exchange chat.	Message saved in DB and broadcast to User B in real time.	Passed
TC-10	Reviews	User submits a 5-star review after exchange completion.	Review saved; average rating on reviewee profile updated.	Passed
TC-11	Safety	User reports another user for inappropriate behavior.	Report record created with Pending moderation status.	Passed

10. Results

LinkWiz was successfully developed, tested, and deployed. The following outcomes were achieved:

- User registration and login function correctly with bcrypt-secured passwords and JWT-based sessions.
- Profile creation and skill management work seamlessly, with skills stored and queried efficiently via MongoDB indexing.
- The smart matching algorithm returns relevant user matches within milliseconds for typical dataset sizes.
- Exchange requests transition smoothly through their full lifecycle: Pending, Accepted, Completed.
- Real-time messaging shows no noticeable latency on local networks; Socket.io room isolation ensures privacy between exchange pairs.
- The personal dashboard accurately aggregates and displays all active, pending, and completed exchanges per user.

- The rating and review system successfully records and displays post-exchange feedback, with live average rating updates.
- The frontend is fully responsive across desktop and mobile viewport sizes using Tailwind CSS.
- Lighthouse performance score exceeded 90 on the key pages: Login, Dashboard, Browse Users.
- All 11 documented test cases passed successfully.

11. Challenges Faced

Challenge	How It Was Resolved
Frontend-backend CORS issues	Configured Express CORS middleware to whitelist the Vercel frontend domain explicitly and aligned API response structures between Axios and Express.
Secure JWT token management	Switched from localStorage to httpOnly cookies for JWT storage and implemented Axios interceptors to handle token expiry and auto-logout.
Real-time status synchronization	Used Socket.io emit events such as accepted and declined to push status changes to both users instantly without requiring page refreshes.
Transitioning from mock data to live DB	Removed hardcoded frontend data incrementally per module and replaced it with Axios API calls backed by real MongoDB records.
React global state management	Used React Context API for authentication state and scoped conversation states locally within the Chat component to avoid unnecessary re-renders.
Responsive UI design	Applied Tailwind CSS responsive prefixes and tested on multiple screen sizes to fix breakpoint-specific layout bugs.

12. Risk Assessment

Risk	Probability	Impact	Mitigation Strategy
MongoDB Atlas free tier storage/connection limits reached	Medium	Medium	Optimize queries, apply compound indexes, monitor Atlas dashboard metrics weekly.
Socket.io performance degradation at scale	Low	Medium	Use rooms efficiently and add Redis adapter for horizontal scaling if the user base grows.
JWT security vulnerabilities / token theft	Low	High	Store tokens in httpOnly cookies, use short expiry times, and rate-limit login attempts.

Risk	Probability	Impact	Mitigation Strategy
Team member unavailability during development	Medium	Medium	Daily standups, clear task ownership, and overlapping skill coverage in GitHub Issues.
Vercel/Render free-tier deployment limits	Low	Low	Monitor usage, keep build sizes minimal, upgrade tier if required.
User safety incidents such as harassment or abuse	Medium	High	Reporting, blocking, message logs, and admin suspension tools.

13. Future Enhancements

Enhancement	Description
Video Calling	Integrate WebRTC-based peer-to-peer video sessions so users can conduct live skill sessions directly inside the platform.
AI Skill Recommendation	Build an AI-powered recommendation engine that suggests skills to learn based on the user's profile, market trends, and community data.
Push Notifications	Add browser push notifications and email alerts for exchange requests, messages, and review reminders.
Mobile Application	Develop a React Native cross-platform mobile app for iOS and Android using the existing REST API backend.
Real-Time Typing Indicators	Enhance the Socket.io messaging layer with typing indicators and read receipts.
Advanced AI Moderation	Integrate NLP-based chat monitoring to automatically flag abusive content.
Group Skill Sessions	Allow multiple users to join a shared learning session.
Skill Verification Badges	Introduce community-vetted skill badges where experienced users can vouch for another user's skill level.

14. Conclusion

LinkWiz successfully achieves its core goal of enabling users to exchange skills through a collaborative, free, and secure online platform. By removing financial barriers and replacing monetary transactions with a knowledge-barter model, the platform creates an ecosystem of mutual learning that benefits students, professionals, freelancers, and lifelong learners alike.

The project demonstrates practical implementation of key software engineering principles: three-tier architecture design, RESTful API development, real-time communication with WebSockets, JWT-based authentication, NoSQL database design, agile sprint planning, and comprehensive testing strategies. The MERN stack proved to be a robust and efficient choice for delivering a modern, responsive, and scalable web application within the defined timeline.

The six-sprint development plan, clear team role assignments, and structured milestone tracking ensured the project was delivered on time with all high-priority features fully implemented. The platform is live, tested, and ready for real-world use, with a solid technical foundation that supports the future enhancements outlined in this report.

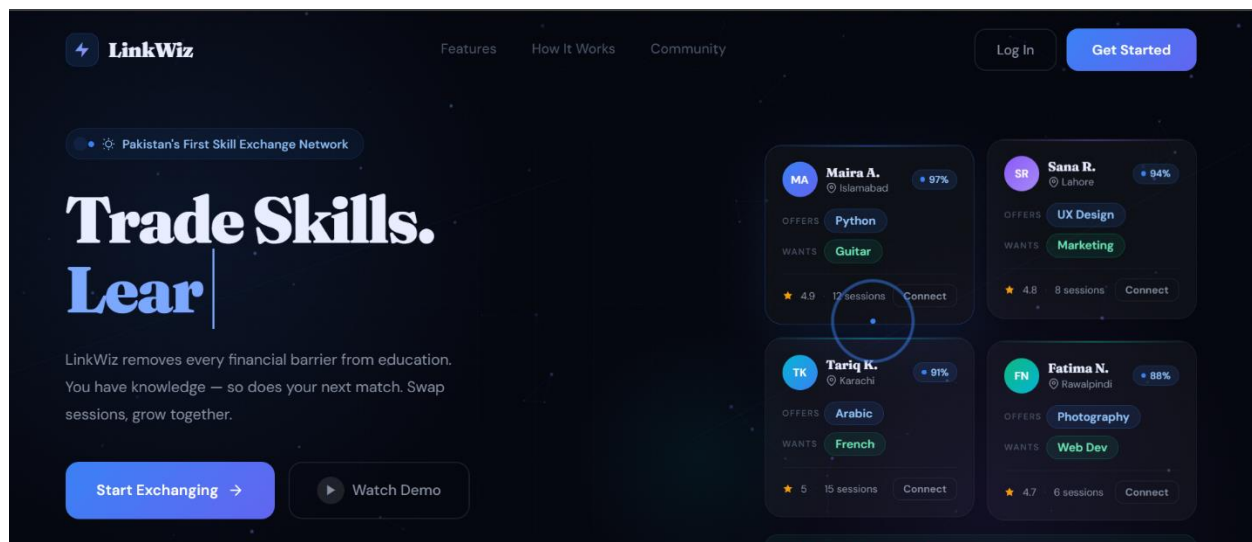
LinkWiz is not just a course project; it is a working solution to a real problem faced by learners across Pakistan and beyond.

15. References

1. React Documentation. Available at <https://react.dev>
2. Node.js Documentation. Available at <https://nodejs.org>
3. Express.js Documentation. Available at <https://expressjs.com>
4. MongoDB Documentation. Available at <https://www.mongodb.com/docs>
5. Mongoose ODM Documentation. Available at <https://mongoosejs.com/docs>
6. JSON Web Tokens Introduction. Available at <https://jwt.io/introduction>
7. bcrypt.js GitHub Repository. Available at <https://github.com/dcodeIO/bcrypt.js>
8. Socket.io Documentation. Available at <https://socket.io/docs>
9. Tailwind CSS Documentation. Available at <https://tailwindcss.com/docs>
10. Sommerville, I. *Software Engineering*, 10th Edition. Pearson Education.

16. Appendix

SCREENSHOTS:



 **LinkWiz**

[Features](#) [How It Works](#) [Community](#)

[Log In](#) [Get Started](#)



Welcome back

Log in to continue your skill exchange journey

EMAIL ADDRESS

you@example.com

PASSWORD

☐ Remember me [Forgot password?](#)

[Log In](#)

or continue with

Google

GitHub

Don't have an account? [Sign up free](#)

 **LinkWiz**

[Features](#) [How It Works](#) [Community](#)

[Log In](#) [Get Started](#)



Create your account

Join thousands of learners exchanging skills across Pakistan

FULL NAME

Sarah Ahmed

CITY

Lahore, Pakistan

EMAIL ADDRESS

you@example.com

PASSWORD

 SKILLS YOU OFFER

e.g. Python, UX Design

Separate with commas

 SKILLS YOU WANT

e.g. Guitar, French

Separate with commas

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#)

[Create Account](#)

 Free forever

 No credit card

 2-min setup

Already have an account? [Log in](#)

LinkWiz

BrowseDashboardIncomingOutgoingMessages

u

YOUR HUB

Welcome back,

You have 4 incoming requests and 2 active exchanges running right now.

SK

Lahore

sessions

0

Incoming

Received requests

View →

1

Outgoing

Sent requests

View →

2

Active

Running exchanges

0

Completed

Finished exchanges

LinkWiz

BrowseDashboardIncomingOutgoingMessages

u

Active Exchanges

View all →

Recent Activity

SR

Sana R.

Lahore · 4.8

Copywriting

UX Design

Progress

40%

Next: Tomorrow, 4 PM

Message

MA

Maira A.

Islamabad · 4.9

Digital Marketing

Python

Progress

20%

Next: Sat, 6 PM

Message

Recent Activity

✓

Sana R. accepted your exchange request

1h ago

New request from Tariq K. — Arabic ↔ Copywriting

3h ago

Session reminder: Sana R. tomorrow at 4 PM

5h ago

✓

Exchange with Ali S. marked complete

2d ago

YOUR SKILLS

SEO

Copywriting

Social Media

+ Add

Browse Users

View Incoming

LinkWiz

BrowseDashboardIncomingOutgoingMessages

u

Browse Community

Find learners and mentors across Pakistan matching your criteria.

Showing 9 members

Filters

Clear All

SKILLS

React

Python

UI/UX

Marketing

SEO

Guitar

CITY

e.g. Lahore, Karachi

AVAILABILITY

☐ Weekends

☐ Weekdays

☐ Evenings

Apply Filters

AR

Ahmed Raza

Lahore

Offers

SEO

Wants

React

5 (0)

View Profile

FK

Fatima Khan

Islamabad

Offers

React

Wants

Marketing

5 (0)

View Profile

AH

Ali Hassan

Karachi

Offers

Python

Wants

Public Speaking

5 (0)

View Profile

SA

Sara Ahmed

Rawalpindi

Offers

Figma

Wants

Web Development

5 (0)

View Profile

LinkWiz

BrowseDashboardIncomingOutgoingMessages

u

EXCHANGE REQUESTS

Incoming Requests

People who want to swap skills with you. Review their offer and decide to accept or pass.

2 pending requestsOldest requests expire in 7 days

RQ

Reena Qureshi

Islamabad

50 sessions

Offers

Graphic Designing Skill Exchange

Wants

Python Skill Exchange

Let's exchange skills

RejectAccept

6/1/2026

ST

Sana Tariq

Rawalpindi

50 sessions

Offers

JavaScript Programming Skill Exchange

Wants

Python Skill Exchange

Let's exchange skills

RejectAccept

6/1/2026

LinkWiz

BrowseDashboardIncomingOutgoingMessages

u

EXCHANGE REQUESTS

Outgoing Requests

Requests you've sent out. Track their status and cancel any pending ones if your plans change.

FilterAll1Pending0Accepted1Rejected0

MM

Maira Malik

Rawalpindi

5

Accepted

You offer

Python Skill Exchange

for

WebDevelopment Skill Exchange

Active exchange

Let's exchange skills

6/1/2026

LinkWiz

BrowseDashboardIncomingOutgoingMessages

u

Active Exchanges

Track all your ongoing skill swap partnerships

4

Total Swaps

All time

2

Active Now

In progress

16

Sessions Done

This month

4.9

Avg Rating

Received

All SwapsActivePendingReview

??

Unknown User

PENDING

Unknown

WebDevelopment Skill Exchange

Python Skill Exchange

Match %

Progress

0/0

Next Session

MM

Maira Malik

PENDING

Rawalpindi

Python Skill Exchange

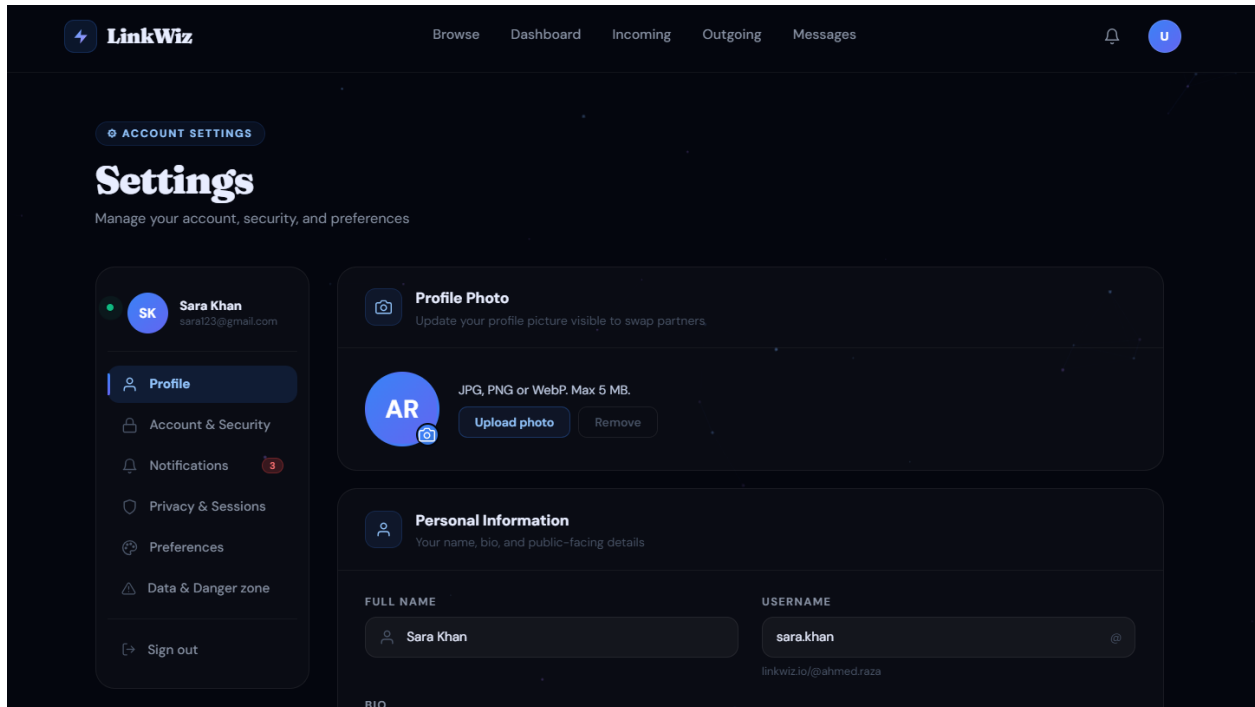
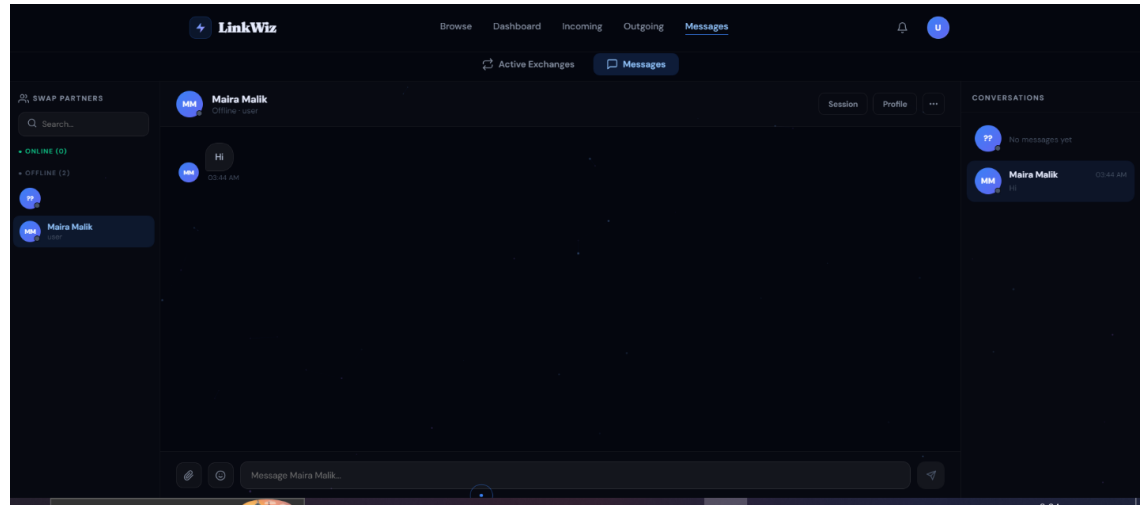
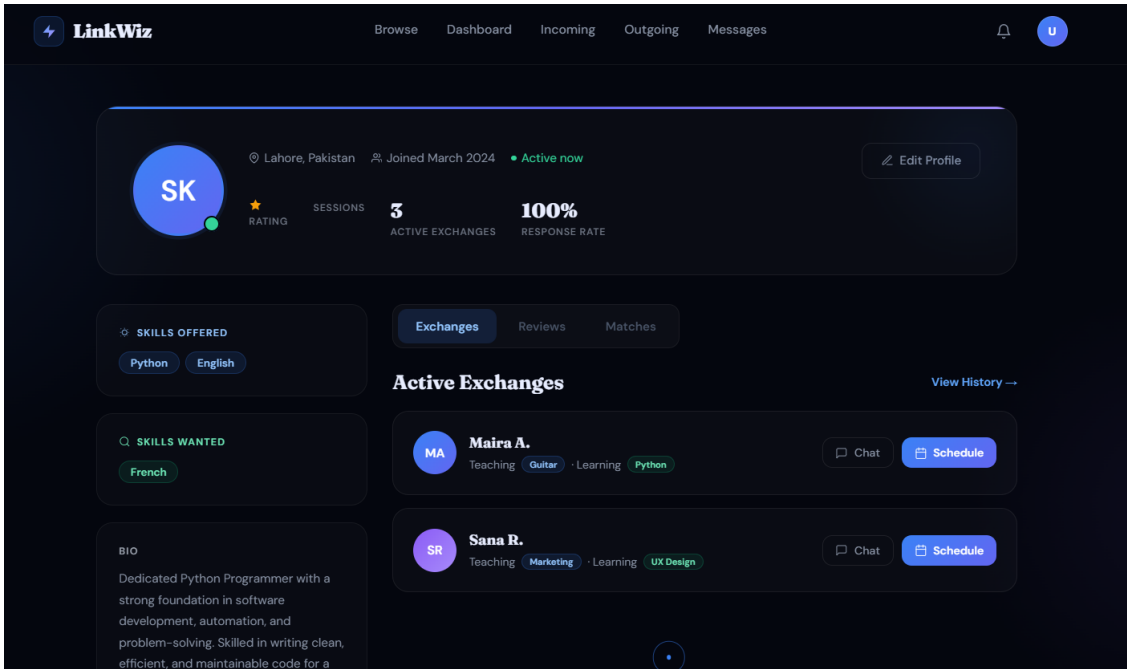
WebDevelopment Skill Exchange

Match %

Progress

0/0

Next Session



 LinkWiz

FeaturesHow It WorksCommunity

Log InGet Started

[← Back to Profile](#)

Edit Profile

Update your information and manage your skill listings.



Sara Khan
JPG, GIF or PNG. Max size of 2MB.
[Change Photo](#)

Basic Information

FULL NAME

CITY

 LinkWiz

FeaturesHow It WorksCommunity

Log InGet Started

Skills You Offer

Search from our skill library or type your own.

Python ×English ×

[Add](#)

TECHNOLOGY

PythonTechnology

JavaScriptTechnology

TypeScriptTechnology

PHPTechnology

Deep LearningTechnology

Cloud ComputingTechnology

Web DevelopmentTechnology

 LinkWiz

FeaturesHow It WorksCommunity

Log InGet Started

Skills You Want to Learn

Search from our skill library or type your own.

French ×

[Add](#)

Availability

☒ Weekday Mornings

☒ Weekday Evenings

☐ Weekends

☐ Flexible

☐ Remote Only

☐ In-Person

[Cancel](#)[Save Changes](#)

ORGANIZATION

Maira's Org - 2026-0...

PROJECT

Project 0

?

linkwiz

+

linkwiz > linkwiz

View monitoring

Visualize your data

Create collection

Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. Document size	Indexes	Total Index size
exchanges	-	36.86 kB	3.64 kB	16	227.00 B	1	36.86
messages	-	36.86 kB	719.00 B	5	143.00 B	1	36.86
reviews	-	36.86 kB	634.00 B	4	158.00 B	1	36.86
services	-	36.86 kB	2.09 kB	10	209.00 B	1	36.86
skills	-	45.06 kB	13.87 kB	171	81.00 B	2	86.02
users	-	36.86 kB	4.76 kB	9	528.00 B	2	73.73

CLUSTERS (1)

Search clusters

linkwiz

admin

linkwiz

exchanges

messages

reviews

Alert System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales